From Data to Solutions · Weekend 7

Fine-tuning methods for LLMs

Adapting a model you did not train

Carlos Cotrini

Section 1

From pretraining to adapting

1

From pretraining to adapting

You will start from a released model

Pretraining Apertus-70B

6.30×10^{24} FLOP

Over 6 million GPU hours

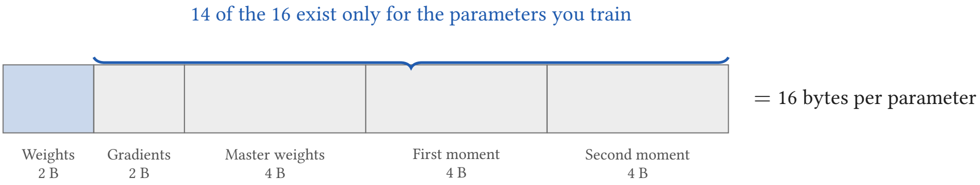
About 4.95 GWh

What you will actually do: take the released model and adapt it

- Two days have priced one pretraining run
- None of it is a decision you will ever make
- You cannot pretrain, so you adapt, and the same ledger prices that

From pretraining to adapting

Sixteen bytes per parameter



- Mixed precision Adam, from this morning: $2 + 2 + 4 + 4 + 4$
- Apertus-8B needs 128.8 GB of state, and a GH200 holds 96 GB
- Freeze a parameter and 14 of its 16 bytes disappear

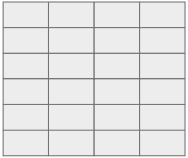
Section 2

One matrix, and a low rank update

2

Twenty-four numbers, or ten

W: 6 outputs $\times$ 4 inputs



Frozen: no gradient, no moments

A: 1×4

$$A = \begin{bmatrix} a_1 & a_2 & a_3 & a_4 \end{bmatrix}$$

B: 6×1

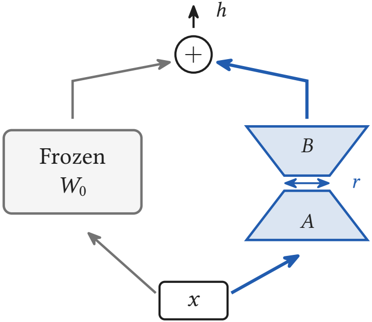
$$B = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \end{bmatrix} \qquad \begin{bmatrix} b_1 a_1 & b_1 a_2 & b_1 a_3 & b_1 a_4 \\ b_2 a_1 & b_2 a_2 & b_2 a_3 & b_2 a_4 \\ b_3 a_1 & b_3 a_2 & b_3 a_3 & b_3 a_4 \\ b_4 a_1 & b_4 a_2 & b_4 a_3 & b_4 a_4 \\ b_5 a_1 & b_5 a_2 & b_5 a_3 & b_5 a_4 \\ b_6 a_1 & b_6 a_2 & b_6 a_3 & b_6 a_4 \end{bmatrix}$$

$\Delta W = BA$

- Full fine-tuning updates all 24
- LoRA freezes W and learns B and A: $6 + 4 = 10$ numbers, a ratio of 2.4
- Every row of BA is the row A, rescaled. That is what rank one means

One matrix, and a low rank update

The adapter is added, not substituted



$$h = \underbrace{W_0 x}_{\text{frozen}} + \underbrace{\frac{\alpha}{r} BAx}_{\text{trained}}$$

- Both paths read the same x, and their outputs are added
- α / r rescales the adapter, and it behaves like a learning rate
- The paper's own tables use ratios of 1, 2 and 8; the PEFT default is $r = 8$, $\alpha = 8$

B starts at zero, A starts random

A random

$$\begin{bmatrix} a & a & a & a \end{bmatrix}$$

$$B = 0 \quad \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

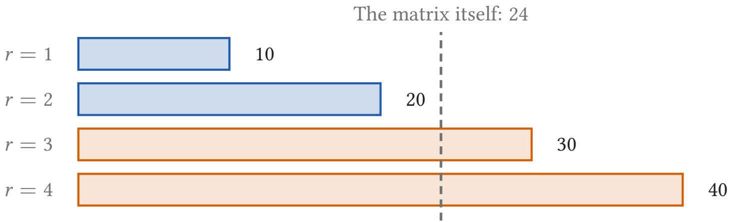
$BA = 0$

$$W_0 + BA = W_0$$

The first forward pass is exactly the pretrained model

- The adapter does nothing at all on step one
- So training starts where pretraining stopped, not somewhere random
- If both started at zero, both gradients would be zero and nothing would ever move

Rank three costs more than the matrix



- Break even at $r = dk/(d + k) = 2.4$ on this matrix
- On a square $d \times d$ matrix that is $r = d/2$, and the saving is $d/(2r)$
- Llama-2-7B's query projection is 4096×4096: at $r = 8$ that is 65 536 numbers against 16 777 216, a factor of 256

Section 3

The same picture at real scale

Rank 16, and 0.08 percent of the model trains

8.05×10^9 parameters

6 815 744 trainable, 0.0846 percent of the model

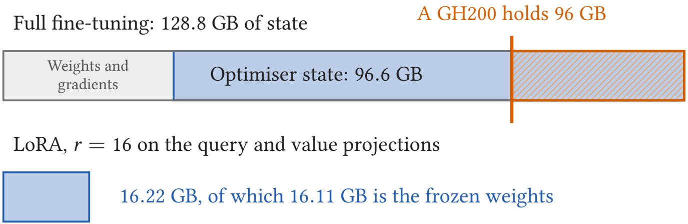
Target modules	Rank	Trainable	Percent
`q_proj`, `v_proj` (the PEFT default)	16	6 815 744	0.0846
All six linear layers	16	39 845 888	0.4948
All six linear layers	64	159 383 552	1.9792
All six linear layers	256	637 534 208	7.9167

- The lower bar is drawn to scale, which is why you cannot see it
- Even rank 256 on all six linear layers stays under 8 percent

Apertus-8B's shape is reconstructed from its published totals: 32 layers, $d_{\text{model}} = 4096$, 8 key and value heads.

The same picture at real scale

128.8 GB does not fit; 16.22 GB does



- Optimiser state: 96.6 GB becomes 0.082 GB
- Total state falls only 7.9 times, because the frozen weights stay
- The adapter file is 13.6 MB, against a 16.1 GB base model
- What needed more than one accelerator this morning now needs one